JS

Array Methods

Complete Class Notes

map · filter · reduce · find · forEach · fill · flat · sort

map() filter() reduce() find() forEach() fill()

Prepared for Classroom Use · JavaScript ES6+

What are Array Methods?

JavaScript Array Methods are built-in functions that help us work with arrays easily. Instead of writing long loops, we use these methods to **transform**, **filter**, **search**, and **reduce** arrays in a clean, readable way.

Quick Reference Table

Method	Returns	Mutates Original?	Use When You Want To…
`map()`	New array (same length)	No ■	Transform every element
`filter()`	New array (shorter)	No ■	Keep only matching elements
`reduce()`	Single value	No ■	Boil down to one result
`find()`	One element / undefined	No ■	Get the first match
`findIndex()`	Index / -1	No ■	Find position of first match
`forEach()`	undefined	No ■	Loop for side effects
`fill()`	Modified array	YES ■■	Fill positions with a value
`some()`	Boolean	No ■	Check if any element passes
`every()`	Boolean	No ■	Check if all elements pass
`flat()`	New array	No ■	Flatten nested arrays
`flatMap()`	New array	No ■	Map then flatten 1 level
`includes()`	Boolean	No ■	Check if value exists
`sort()`	Modified array	YES ■■	Sort elements in place
`push()`	New length (number)	YES ■■	Add element(s) to end
`pop()`	Removed element	YES ■■	Remove last element
`unshift()`	New length (number)	YES ■■	Add element(s) to beginning
`shift()`	Removed element	YES ■■	Remove first element
`indexOf()`	Index number or -1	No ■	Find position of a value

1. map() — Transform Every Element

map() creates a **new array** by applying a function to each element. The original array is **never changed**. The result always has the **same length**.

Syntax

```
array.map(callbackFn(element, index, array))
```

Example 1 — Double every number

```
// Original array
const nums = [1, 2, 3, 4, 5];

// map() transforms each element
const doubled = nums.map(n => n * 2);
// → [2, 4, 6, 8, 10]   (new array, same length!)
```

Example 2 — Extract names from objects

```
const students = [{name:"Arya", marks:85}, {name:"Ravi", marks:72}];
const names = students.map(s => s.name);
// → ["Arya", "Ravi"]
```

Example 3 — Add GST to prices

```
const prices = [100, 200, 500];
const withGST = prices.map(p => +(p * 1.18).toFixed(2));
// → ["118.00", "236.00", "590.00"]
```

■ **map() always returns an array of the SAME LENGTH. Use it to transform, not to filter.**

2. filter() — Keep Matching Elements

filter() creates a **new array** with only the elements that pass the test. If no elements pass, it returns an **empty array []** — never undefined.

Example 1 — Only even numbers

```
const nums = [1,2,3,4,5,6];
const evens = nums.filter(n => n % 2 === 0);
// → [2, 4, 6]
```

Example 2 — Students who passed (marks >= 40)

```
const students = [
  {name:"Arya", marks:85},
  {name:"Ravi", marks:33},
  {name:"Priya", marks:72}
];

const passed = students.filter(s => s.marks >= 40);
// → [{name:"Arya",marks:85}, {name:"Priya",marks:72}]
```

Example 3 — Remove empty strings

```
const items = ["apple", "", "mango", "", "banana"];
const clean = items.filter(item => item !== "");
// → ["apple", "mango", "banana"]
```

■ **filter() vs find(): filter() returns an ARRAY of ALL matches. find() returns ONE element.**

3. reduce() — Boil Down to One Value

reduce() processes each element and accumulates a single result. It can produce a **number, string, object, or even another array**. This is the most powerful array method!

Syntax

```
array.reduce(callbackFn(accumulator, current, index), initialValue)
```

Example 1 — Sum of all numbers

```
const nums = [10, 20, 30, 40];
const sum = nums.reduce((acc, curr) => acc + curr, 0);
// → 100
```

Example 2 — Count fruit occurrences

```
const fruits = ["apple", "mango", "apple", "mango", "apple"];
const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
// → { apple: 3, mango: 2 }
```

Example 3 — Total cart price

```
const cart = [{item:"Pen",price:20}, {item:"Book",price:150}];
const total = cart.reduce((sum, p) => sum + p.price, 0);
// → 170
```

■ **ALWAYS provide initialValue (2nd arg)! Without it, reduce crashes on empty arrays.**

4. find() — Get the First Match

find() returns the **first element** that passes the test and stops immediately. Returns **undefined** if nothing is found.

```
const nums = [1, 3, 4, 6, 7];
const firstEven = nums.find(n => n % 2 === 0);
// → 4   (only the FIRST match, not [4,6])

const users = [{id:1,name:"Arya"}, {id:2,name:"Ravi"}];
const user = users.find(u => u.id === 2);
// → {id:2, name:"Ravi"}
```

5. findIndex() — Get the Position

Like find(), but returns the **index** (position number) instead. Returns **-1** if not found.

```
const nums = [10, 20, 30, 40];
const idx = nums.findIndex(n => n > 25);
// → 2   (30 is at index 2)
```

6. forEach() — Loop for Side Effects

forEach() runs a function for each element but **returns undefined**. Use it for logging, updating DOM, or any action where you don't need a return value.

```
const fruits = ["Apple", "Mango", "Banana"];

fruits.forEach((fruit, index) => {
  console.log(`${index+1}. ${fruit}`);
});

// Output:
// 1. Apple
// 2. Mango
// 3. Banana
```

■ **forEach() cannot be stopped with break or return. Use a for loop or some() if you need early exit.**

7. fill() — Fill with a Value

Fills elements with a static value. **Mutates** the original array!

```
const arr = [1,2,3,4,5];
arr.fill(0);                  // → [0,0,0,0,0]
arr.fill(9, 2, 4);          // → [0,0,9,9,0]  (index 2 to 3)

// Common trick — create array of N zeros:
const zeros = new Array(5).fill(0);  // → [0,0,0,0,0]
```

8. some() & every() — Boolean Checks

some()	every()
Returns **true** if AT LEAST ONE element passes	Returns **true** only if **ALL** elements pass

```
const marks = [45, 78, 35, 90];

marks.some(m => m >= 75);  // → true  (78 and 90 pass)
marks.every(m => m >= 35); // → true  (all pass)
marks.every(m => m >= 50); // → false (35 fails!)
```

9. flat() — Remove Nesting

```
const nested = [1, [2,3], [4,[5]]];
nested.flat();      // → [1,2,3,4,[5]]  (1 level)
nested.flat(2);     // → [1,2,3,4,5]   (2 levels)
nested.flat(Infinity); // → [1,2,3,4,5]   (all levels!)

// flatMap() = map() + flat(1) combined:
const words = ["hello world", "foo bar"];
words.flatMap(w => w.split(" ")); // → ["hello","world","foo","bar"]
```

10. sort() — Sort In Place

```
// ■ Wrong — sorts as strings by default:
[10,2,1,20].sort();           // → [1,10,2,20]  WRONG!

// ■ Correct — use compare function:
[10,2,1,20].sort((a,b) => a-b);   // → [1,2,10,20] ASC
[10,2,1,20].sort((a,b) => b-a);   // → [20,10,2,1] DESC
```

■ **sort() MUTATES the original array! Use [...arr].sort() to sort a copy safely.**

Add & Remove Methods — push, pop, shift, unshift

These four methods **mutate (change) the original array**. They add or remove elements from either end of the array. Very commonly used in real projects!

Method	Where	Action	Returns	Mutates?
push()	END →	Add one or more elements	New length	YES
pop()	← END	Remove last element	Removed element	YES
unshift()	→ START	Add one or more at beginning	New length	YES
shift()	START ←	Remove first element	Removed element	YES

11. push() — Add to End

```
const fruits = ["Apple", "Mango"];

fruits.push("Banana");
// fruits → ["Apple", "Mango", "Banana"]  (added at END)

// Push multiple items at once:
fruits.push("Grapes", "Kiwi");
// fruits → ["Apple","Mango","Banana","Grapes","Kiwi"]

// push() returns the new LENGTH:
const newLen = fruits.push("Papaya"); // → 6
```

12. pop() — Remove from End

```
const fruits = ["Apple", "Mango", "Banana"];

const removed = fruits.pop();
// removed → "Banana"   (the item that was removed)
// fruits  → ["Apple", "Mango"]   (Banana is gone!)

// Common use: Stack (LIFO) — Last In First Out
const stack = [];
stack.push("page1"); stack.push("page2"); stack.push("page3");
stack.pop(); // → "page3"  (back navigation!)
```

13. unshift() — Add to Beginning

```
const nums = [3, 4, 5];

nums.unshift(1, 2);
// nums → [1, 2, 3, 4, 5]   (1 and 2 added at START)

// unshift() also returns new length:
const len = nums.unshift(0);  // → 6
```

14. shift() — Remove from Beginning

```
const queue = ["Arya", "Ravi", "Priya"];

const first = queue.shift();
// first → "Arya"              (the item that was removed)
// queue → ["Ravi", "Priya"]   (Arya is served, removed!)

// Common use: Queue (FIFO) — First In First Out
// Like a ticket counter — serve the first person in line
```

■ **Memory trick: push/pop = END of array. shift/unshift = BEGINNING (start). pop & shift REMOVE. push & unsh**

Search Methods — includes & indexOf

These two methods help you **search** for a value inside an array. They are simple, fast, and use **strict equality (===)** to compare.

15. includes() — Does it Exist?

includes() checks if an array **contains a specific value**. Returns **true** or **false**. Simple and readable!

```
const colors = ["red", "green", "blue"];

colors.includes("green");   // → true
colors.includes("yellow");  // → false

// Real-world use: Check if user has permission
const roles = ["admin", "editor", "viewer"];
if (roles.includes("admin")) {
  console.log("Access granted!");
}

// Also works with numbers:
const allowedCodes = [200, 201, 204];
allowedCodes.includes(404);  // → false
allowedCodes.includes(200);  // → true
```

16. indexOf() — Where is it?

indexOf() returns the **index (position)** of the first occurrence of a value. Returns **-1** if the value is not found. Great for checking position AND existence.

```
const fruits = ["Apple", "Mango", "Banana", "Mango"];

fruits.indexOf("Mango");    // → 1   (first occurrence at index 1)
fruits.indexOf("Banana");   // → 2
fruits.indexOf("Papaya");   // → -1  (not found!)

// Check existence using indexOf:
if (fruits.indexOf("Mango") !== -1) {
  console.log("Mango found!");
}

// indexOf with start position (search from index 2):
fruits.indexOf("Mango", 2);  // → 3  (skips index 0,1,2)
```

includes() vs indexOf() — Which to Use?

includes()	indexOf()
Returns **boolean** (true/false)	Returns **number** (index or -1)
Use when: just checking **if exists**	Use when: need the **position** too
arr.includes("x") → true/false	arr.indexOf("x") → 0,1,2... or -1
Can detect **NaN** values	Cannot detect NaN (limitation)

■ **Modern code prefers includes() for readability. indexOf() is used when you need the exact position.**

All 6 Methods — Side by Side

```
const arr = [10, 20, 30];

// ■■ ADD ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
arr.push(40);         // arr = [10,20,30,40]   added at END
arr.unshift(0);       // arr = [0,10,20,30,40] added at START

// ■■ REMOVE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
arr.pop();           // arr = [0,10,20,30]   removed from END
arr.shift();         // arr = [10,20,30]     removed from START

// ■■ SEARCH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
arr.includes(20);    // → true  (does it exist?)
arr.indexOf(20);     // → 1     (at which position?)
```

Method Chaining — The Real Power

Since map(), filter(), and most methods return a new array, you can **chain** them together to write powerful, readable data pipelines without any loops.

Example — Total marks of students who passed, sorted descending

```
const students = [
  {name:"Arya",  marks:85},
  {name:"Ravi",  marks:33},    // failed
  {name:"Priya", marks:70},
  {name:"Sam",   marks:91}
];

const totalPassMarks = students
  .filter(s => s.marks >= 40)     // Step 1: keep passed only
  .map(s => s.marks)              // Step 2: extract marks
  .reduce((a,b) => a+b, 0);       // Step 3: sum them up

// → 246   (85 + 70 + 91, Ravi excluded)
```

Remove Duplicates (Common Interview Trick)

```
const arr = [1,2,2,3,3,4];

// Method 1: Set + spread (clean, modern)
const unique1 = [...new Set(arr)];
// → [1, 2, 3, 4]

// Method 2: filter + indexOf
const unique2 = arr.filter((val,idx) => arr.indexOf(val) === idx);
// → [1, 2, 3, 4]
```

Interview Questions

Q 1. **What is the difference between map() and forEach()?**

Ans: map() returns a NEW array of transformed values. forEach() always returns undefined — it's only for side effects like logging or updating the DOM. You cannot chain forEach().

Q 2. **What is the difference between find() and filter()?**

Ans: find() returns the FIRST element that passes the test (one element or undefined). filter() returns ALL matching elements in a new array (could be empty []).

Q 3. **Why should you always pass an initialValue to reduce()?**

Ans: Without initialValue, reduce() crashes with TypeError on empty arrays. With initialValue (like 0 or {}), it safely returns the initial value when the array is empty.

Q 4. **Which array methods mutate (change) the original array?**

Ans: Mutating methods: sort(), fill(), splice(), push(), pop(), shift(), unshift(), reverse(). Safe methods that return new arrays: map(), filter(), flat(), flatMap(), slice().

Q 5. **Can you implement map() using reduce()?**

Ans: Yes! arr.reduce((acc, curr) => { acc.push(curr * 2); return acc; }, []) — reduce can implement map, filter, and more. This is a classic interview question.

Q 6. **What does flat(Infinity) do?**

Ans: It completely flattens an array of any nesting depth into a single flat array. flat() with no argument only flattens 1 level by default.

Q 7. **How do you find the maximum value in an array without Math.max?**

Ans: arr.reduce((max, curr) => curr > max ? curr : max, arr[0]) — use reduce to track the running maximum across all elements.

Practice Problems

Solve these problems using only array methods — no for/while loops allowed! Write the answers in your notebook and verify by running in browser console.

Easy Q1 — Filter & Map

Given [3, 7, 2, 9, 1, 5, 8], use filter() to keep numbers greater than 5, then use map() to square each of those numbers.

Hint: Expected output: [49, 81, 64] (7^2, 9^2, 8^2)

Your Answer:

Easy Q2 — reduce() Average

Given students [{name:'Arya',marks:85}, {name:'Ravi',marks:60}, {name:'Priya',marks:72}, {name:'Sam',marks:91}], use reduce() to calculate the average marks.

Hint: Expected output: 77 (308 ÷ 4)

Your Answer:

Medium Q3 — Chain Methods

Given products [{name:'Pen',price:20,inStock:true}, {name:'Book',price:150,inStock:false}, {name:'Bag',price:500,inStock:true}], filter only in-stock items, apply 10% discount with map(), then use reduce() to get the total price.

Hint: Expected output: 468 ((20×0.9) + (500×0.9))

Your Answer:

Medium

Q4 — flatMap()

Given ["the quick brown fox", "jumps over the lazy dog"], use flatMap() to get a flat array of all individual words, then filter() to keep only words with 4 or more characters.

Hint: Expected output: ["quick", "brown", "jumps", "over", "lazy"]

Your Answer:

Hard

Q5 — Group By Grade

Use reduce() to group students by grade: A (≥80), B (60–79), C (below 60). Input: [{name:'Arya',marks:85}, {name:'Ravi',marks:60}, {name:'Priya',marks:45}, {name:'Sam',marks:91}, {name:'Tanya',marks:73}]

Hint: Expected output: { A: ["Arya","Sam"], B: ["Ravi","Tanya"], C: ["Priya"] }

Your Answer:

Final Cheat Sheet — One Page Summary

Method	What it does	Mutates?
map()	Transform every element → new array (SAME length)	No
filter()	Keep matching elements → new array (shorter)	No
reduce()	Boil down all elements to ONE value	No
find()	First matching element OR undefined	No
findIndex()	Index of first match OR -1	No
forEach()	Loop for side effects — returns undefined	No
fill()	Fill positions with a value	YES ■
some()	true if AT LEAST ONE element passes	No
every()	true if ALL elements pass	No
flat()	Flatten nested arrays by N levels	No
flatMap()	map() + flat(1) in one step	No
includes()	Check if a value exists → boolean	No
sort()	Sort in place — always use compare fn for numbers!	YES ■
push()	Add element(s) to END → returns new length	YES ■
pop()	Remove last element ← returns removed item	YES ■
unshift()	Add element(s) to BEGINNING → returns new length	YES ■
shift()	Remove first element ← returns removed item	YES ■
indexOf()	Position of first match → index number or -1	No

Golden Rules to Remember

1. map() → transform (same length always) | filter() → select (can be shorter) | reduce() → one result
2. Always pass initialValue to reduce() to avoid crashes on empty arrays
3. sort() and fill() MUTATE the original. All others are safe.
4. forEach() returns undefined — never use it when you need the result
5. Chain methods: filter().map().reduce() is clean, readable, and powerful